

Problem name: Banker's Algorithm

Source code:

```
#include <iostream>

#include <vector>

using namespace std;

int main() {

    int numProcesses, numResources;

    cout << "Enter number of processes: ";

    cin >> numProcesses;

    cout << "Enter number of resources: ";

    cin >> numResources;

    vector<vector<int>> allocation(numProcesses, vector<int>(numResources));

    vector<vector<int>> maxDemand(numProcesses, vector<int>(numResources));

    vector<int> available(numResources);

    cout << "\nEnter Allocation Matrix :\n";

    for (int i = 0; i < numProcesses; i++) {

        for (int j = 0; j < numResources; j++) {

            cin >> allocation[i][j];

        }

    }

    cout << "\nEnter Maximum Demand Matrix :\n";
```

```

for (int i = 0; i < numProcesses; i++) {
    for (int j = 0; j < numResources; j++) {
        cin >> maxDemand[i][j];
    }
}

cout << "\nEnter Available Resources : ";

for (int j = 0; j < numResources; j++) {
    cin >> available[j];
}

vector<vector<int>> need(numProcesses, vector<int>(numResources));

for (int i = 0; i < numProcesses; i++) {
    for (int j = 0; j < numResources; j++) {
        need[i][j] = maxDemand[i][j] - allocation[i][j];
    }
}

vector<int> finished(numProcesses, 0);

vector<int> safeSequence;

for (int k = 0; k < numProcesses; k++) {
    for (int i = 0; i < numProcesses; i++) {
        if (finished[i] == 0) {
            bool canExecute = true;

            for (int j = 0; j < numResources; j++) {
                if (need[i][j] > available[j]) {

```

```

        canExecute = false;

        break;
    }
}

if (canExecute) {
    safeSequence.push_back(i);

    for (int j = 0; j < numResources; j++) {
        available[j] += allocation[i][j];
    }

    finished[i] = 1;
}
}
}

if(safeSequence.size()<numProcesses){
    cout<<"There exists a deadloack";

    return 0;
}

cout << "\nThe SAFE Sequence is:\n";
for (int i = 0; i < numProcesses; i++) {
    cout << "P" << safeSequence[i]+1;

    if (i != numProcesses - 1)
        cout << " -> ";
}

cout << endl;

```

```
    return 0;  
}
```

Output:

```
Enter number of processes: 5  
Enter number of resources: 3  
  
Enter Allocation Matrix :  
0 1 0  
2 0 0  
3 0 2  
2 1 1  
0 0 2  
  
Enter Maximum Demand Matrix :  
7 5 3  
3 2 2  
9 0 2  
4 2 2  
5 3 3  
  
Enter Available Resources : 3 3 2  
  
The SAFE Sequence is:  
P2 -> P4 -> P5 -> P1 -> P3
```